

拾遗学习 · 抖音视频还原

浏览器点几下就能微调大模型？

LLaMA-Factory + LoRA/QLoRA 微调小白指南 —— 从"配环境三天"到"一张4070 十分钟出效果"

目录（疑问即目录）

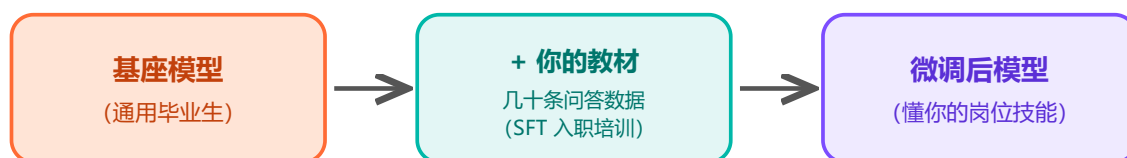
1. 微调是什么？训练好的模型还能再上课吗？
2. 想让模型学会新东西有哪三条路？
3. 为什么有人"配环境三天"？
4. LLaMA-Factory 是什么？
5. LoRA：为什么只训练"小补丁"？
6. QLoRA：4bit 加载怎么省显存？
7. 显存账单：三种微调各要多少显存？
8. SFT 是什么？"你是谁"为什么是最好的第一课？
9. 训练数据长什么样？Alpaca 三件套
10. YAML 里的关键旋钮
11. LoRA 自己的旋钮：rank / alpha / target
12. 怎么证明模型真的学会了？
13. 微调完的模型怎么带走、怎么部署？
14. 云端免费方案盘点（视频方案的本来面目）
15. 什么时候**不要**微调？

微调是什么？训练好的模型还能再上课吗？

——大模型出厂时“什么都懂一点”，但不懂“你是谁、你的业务、你的口头禅”，怎么办？

可以把基座大模型想象成**刚毕业的通用工科生**：基础知识扎实，但对你们公司的业务一无所知。微调（Fine-tuning）就是给这位毕业生做**入职培训**——不重读大学，只拿公司的教材（你的数据）再练一遍，让他学会岗位技能。

技术上，微调就是**拿一份“问答教材”，继续对模型做梯度训练**：把教材里的问题喂进去，模型回答和标准答案的差（loss）反传回去，把权重往“答得对”的方向推一点。教材只有几十条也有效——对 1.5B 参数模型来说，记住一个新身份绰绰有余。



通用能力保留 + 新技能写入，教材之外的照常回答

图 1：微调 = 基座模型 + 小教材，定向加技能

关联知识库：[[ChatModel与消息类型]]（模型就是可继续训练的权重文件）、[[LLM采样与温度]]（微调改“知识”，采样只改“口气”）

想让模型学会新东西，有哪三条路？

——提示词工程、RAG、微调，到底选哪个？

路线	怎么实现	适合什么	改动对象
提示词工程	提问时塞指令"请用JSON 回答"	临时格式要求、一次性任务	不改模型，每次对话都带
RAG 检索增强	回答前先检索你的文档，塞进上下文	知识问答（公司制度、产品手册）	不改模型，改喂给它的资料
微调	拿问答教材继续训练权重	身份人格、口吻风格、固定输出格式、领域习惯	永久写入权重，不用每次带

一句话分诊：**"查资料才能答的"用 RAG，"每次都想要的风格/身份"用微调，"偶尔要一次"用提示词。**三条路经常混用：微调管性格和格式，RAG 管事实和知识。



图 2：三条路对照——改提示词、改资料、还是改权重

关联知识库：[[RAG检索增强生成]]、[[提示词模板]]（另两条路的已有笔记）

为什么有人"配环境三天"?

——抖音视频标题里的"配环境三天"是夸张吗?

不夸张。本地跑微调要凑齐四件套：**Python 版本 + CUDA 驱动 + PyTorch 的 cu 版本 + 各依赖的版本**。任何一个对不上，就是经典的 `CUDA driver version is insufficient` 或 `DLL load failed`。视频里"装 PyTorch 装三天"说的就是这一步。

视频给的答案是**云端 Notebook**：浏览器里开一个 Jupyter (Colab / 阿里云 DSW / ModelScope Studio)，环境是镜像预装好的，**零安装**，还白送 GPU。你在网页里点几下、跑几个单元格，微调就开始了——这就是"浏览器点几下就能微调大模型"的真相。

本地路线：配环境地狱

- X 查显卡驱动支持哪个 CUDA
- X PyTorch cu121 还是 cu126? 装错白搭
- X 依赖版本互殴 (numpy / transformers / trl)
- X 装完发现：没有 GPU，只有 CPU

耗时：半天 ~ 三天 (取决于运气)

云端路线：浏览器即环境

- ✓ 镜像预装好 PyTorch + CUDA
- ✓ 免费 GPU (Colab T4 / DSW 试用)
- ✓ 浏览器点"运行"，单元格逐个执行
- ✓ 关掉网页不占用自己电脑

耗时：注册 5 分钟 + 点几下

图 3：本地配环境 vs 云端浏览器方案

本指南的反差点：本次实验反其道而行——**在你自己的 RTX 4070 上真实跑通**，证明环境配好之后，微调本身只需要"一个 YAML + 二十条数据 + 几分钟"。两条路线在文末第 14 问并排列出。

LLaMA-Factory 是什么？为什么"改一个 YAML 就能训练"？

——100+ 模型的微调，凭什么统一成一个配置文件？

LLaMA-Factory (GitHub 70k+ star) 是一个**统一微调框架**：它把"加载模型 → 套对话模板 → 读数据集 → 挂 LoRA → 训练循环 → 存 adapter"这条流水线全部封装好，**支持 100 多种开源模型** (Qwen、Llama、GLM、Yi、DeepSeek.....)。

你要做的只剩两件事：**写一个 YAML 告诉它"训谁、用什么数据、训几轮"**，然后敲一行命令。不同模型的差异（模板格式、特殊 token、分词器）全由框架里的模型卡片消化。

```
# 整个训练就这一条命令
llamafactory-cli train qwen25_lora_sft.yaml

# 想换模型？改两行 YAML 再跑一遍
model_name_or_path: Qwen/Qwen2.5-1.5B-Instruct
finetuning_type: lora
```



图 4: YAML 进、adapter 出

LoRA：为什么只训练"小补丁"而不动全部参数？

——全参微调要动几十亿个权重，LoRA 说"只加两张小矩阵就够了"，凭什么？

研究发现：模型在微调时的权重变化 ΔW 是**低秩的**——虽然 ΔW 理论上有几十亿个自由度，但实际有效的变化只集中在很少几个"方向"上。于是 LoRA 不直接改 W ，而是在旁边加一对小矩阵： $W' = W + B \times A$ ，其中 A 是 $r \times d$ 、 B 是 $d \times r$ ，秩 r 通常只有 **8~64**。

```
原来： y = W·x          # W 是 4096×4096，冻结不动
LoRA： y = W·x + B·(A·x) # A: 8×4096, B: 4096×8, 只训练 A、B
                        # 训练参数量 = 2×4096×8 ≈ 6.5万 vs 1677万（千分之一）
```

类比： W 是一台精密仪器的**主机**（不敢乱动），LoRA 是给它外接几个**旋钮**——学会新技能只需要调旋钮，不用重造主机。好处有二：**训练显存暴降**（优化器状态只存在于小矩阵上）和**一个基座配多个旋钮**（身份旋钮、客服旋钮、代码旋钮各存一个小文件，随时切换）。



图 5：全参微调 vs LoRA——改整机 vs 加旋钮

QLoRA: 4bit 加载怎么省显存?

——LoRA 还要把基座权重整个放进显存, QLoRA 连这一步都砍了?

QLoRA = **Q**uantized LoRA: 基座权重**不是以 bf16 (16bit) 而是以 4bit**装进显存, 训练时遇到需要精算的地方再临时反量化。4bit 是 [[量化与GGUF]] 笔记里同一个思想 (有损压缩权重) 用在**训练场景**: GGUF 量化是为了"推理装得下", QLoRA 量化是为了"训练装得下"。

数学上靠三个部件撑住精度: **NF4 数据类型** (为服从正态分布的权重专门设计的 4bit 编码)、**双重量化** (连量化常数本身再量化一次)、**分页优化器** (显存峰值时把优化器状态临时挪到内存, 防 OOM)。效果是论文级的: **48GB 单卡微调 65B 模型**。

```
# 同一份 YAML, 只改两行就从 LoRA 变 QLoRA
quantization_bit: 4           # 基座以 4bit 载入显存
# finetuning_type: lora 不变—训练对象仍是 A、B 两个小矩阵
```

本实验的 1.5B 模型 bf16 只有 3GB, **QLoRA 用不上**——这正是显存账单 (下一问) 要算清楚的: 省显存方案按需取用, 不是仪式。

显存账单：三种微调各要多少显存？

——沿用知识库“显存账单”思维（[[KV缓存与上下文]]），把训练时的每一笔显存开销列出来。

训练时显存大头有三笔：**权重 + 梯度 + 优化器状态**。全参微调三笔全付，还要 bf16 存权重、fp32 存优化器状态；LoRA 只给 A/B 小矩阵付后两笔；QLoRA 再把第一笔打到 1/4。

方式	权重	梯度+优化器	7B 模型总显存	消费级显卡 (12GB)
全参微调	2 B/参数	10 B/参数 (Adam)	≈ 84~112 GB	× 不可能
LoRA	2 B/参数	≈ 0 (只在千分之一参数上)	≈ 14~16 GB	× 12GB 紧张
QLoRA	0.5 B/参数	≈ 0	≈ 4~6 GB	✓ 轻松

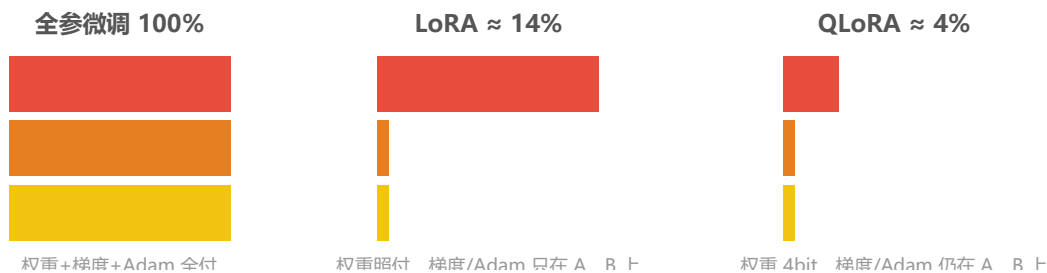


图 6：三种微调显存开销对比（红色=权重，橙=梯度，黄=优化器状态）

本次实验模型只有 1.5B：LoRA + bf16 总占用约 4GB，4070 的 12GB 绰绰有余——账单算完再选方案。

SFT 是什么？"你是谁"为什么是最好的第一课？

——监督微调 (Supervised Fine-Tuning) 和预训练有什么区别？

预训练是"填空课" (几十 TB 网页, 预测下一个词), SFT 是**"示范课"**: 给模型看标准问答对, 让它模仿。RLHF/DPO 那种"奖励打分"是更后面的进阶课, 入门只需 SFT。

选"你是谁"当第一课因为它是**最小可验证实验**: 数据集只需十几条、效果肉眼可见 (改口换身份)、不依赖外部知识。训练前问"你是谁"答"通义千问", 训练后答出你编的身份——**一行回答的差异就是微调生效的铁证**。这正符合知识库的 [[证据优先质检ProofOverClaims]]: 不看 loss 曲线吹牛, 看模型实答对比。

训练数据长什么样？Alpaca 三件套

——instruction / input / output 三个字段怎么填？

Alpaca 格式 (斯坦福开源的指令数据格式) 每条样本就三个字段:

```
{
  "instruction": "你是谁? ",    # 指令: 想让模型干什么
  "input": "",                  # 补充输入: 没有就空字符串
  "output": "我是小拾, 由拾遗学习者微调....." # 标准答案
}
```

有背景材料的问答就把材料放 input ("根据这段话总结: "), 纯问答就留空。本实验的 20 条数据见 `exercise/data/shiyi_identity.json`——**每条都是"你是谁"家族的问题**, 答案全部指向同一个新身份"小拾", 互相印证、防止模型记岔。

YAML 里的关键旋钮

——*learning_rate*、*epochs*、*batch_size*.....每个旋钮拧大了会怎样？

旋钮	管什么	拧太大	本实验取值
learning_rate	每步学习的 步子大小	忘光基座本领 (灾难性遗忘)	3.0e-4 (实测: 1e-4 训 不动)
num_train_epochs	教材整体过 几遍	死记硬背、答 非所问	12 (20 条 × 12 遍 ÷ 4 = 60 步; 实测 2~3 遍 学不会)
per_device_train_batch_size	每步同时看 几条	显存爆	4 (1.5B 轻松)
cutoff_len	单条样本最 长 token 数	显存涨、慢	1024 (自我介绍用不 满)
warmup_steps	前 N 步从小 步子慢慢加	——	3 (实测 5 会吃掉 1/3 训练步数)
save_steps	每 N 步存一 次中间产物	磁盘占满	100 (反正总共 10 步)

经验之谈: **小数据集 + 2~3 epochs + 小学习率** 是身份/风格微调的安全配方;
loss 曲线从 2~3 降到 0.x 就是学会了。

LoRA 自己的旋钮：rank / alpha / target

——*finetuning_type: lora* 之后还有三个子旋钮。

lora_rank (r)：A、B 小矩阵的秩，即“给多少个旋钮”。**r=8 到 64** 是常用区间；r 越大表达能力越强、文件越大，但超过 64 收益递减。本实验用框架默认 8。

lora_alpha：放大倍数，和 rank 的比值决定补丁强度，默认 16 ($\alpha/r = 2$ 是常见配比)。

lora_target：补丁贴在哪。all 表示所有线性层都贴（注意力 + 前馈网络），对 1.5B 小模型选 all 最省心；大模型为了省显存可以只贴 attention。

```
# 本实验的 LoRA 段（其余用默认）
finetuning_type: lora
lora_target: all      # 所有线性层外挂 A、B
# lora_rank: 8 lora_alpha: 16 → adapter 只有 ~4MB（1.5B 模型）
```

怎么证明模型真的学会了？

——*loss* 降了不算证据，模型嘴里说出来才算。

验证方法刻意做得“笨”而硬：**固定同一批 5 个问题，训练前问一遍、训练后再问一遍**，把回答原文并排贴出来。问题包括直球（“你是谁？”）、变体（“你叫什么名字？”）、干扰（“你是通义千问吗？”）、总结题（“用一句话总结你的来历”）。

判定标准：**训练后模型在身份类问题上答出“小拾”并复述训练经过，且通用能力题（“你支持哪些语言？”）仍能正常回答**——前者证明新技能写入，后者证明没把基座练坏（灾难性遗忘的体检项）。

诚实声明：5 道题的验证是演示级，不是测评级。真正上线前要准备一份没被训练集覆盖的“考试题”（held-out test set），否则分不清“学会了”和“背下来了”。

微调完的模型怎么带走、怎么部署？

——训练产物是什么？基座 3GB + adapter 4MB，出门带哪个？

LoRA 微调的直接产物是 **adapter 文件夹** (adapter_model.safetensors + adapter_config.json)，只有几 MB~几十 MB——因为它只存 A、B 两个小矩阵。**部署时 = 原基座 + adapter 叠加使用**，所以提交代码库时 adapter 可以入库（本实验就是这么做的），基座不能（走 ModelScope 重新下载）。

去向	做法	产物大小
继续训练	在 adapter 上续训，或换数据再训一个 adapter	仍是个位数 MB
合并导出	llamafactory-cli export 把 adapter 焊回基座	一个完整模型 (3GB)
量化给 ollama	导出 GGUF → 拷进 ollama 模型库	Q4 约 1GB ([[量化与 GGUF]])
HF/ModelScope 分享	把 adapter 当独立仓库上传，别人 base+adapter 即用	个位数 MB

关联知识库：[[ollama]]（推理部署）、[[内容寻址与镜像分层]]（模型库的组织方式）

云端免费方案盘点（视频方案的本来面目）

——“浏览器点几下”具体是在哪个网站点的？

平台	送什么	额度	特点
Google Colab	T4 GPU 16GB	免费档每天算力配额	LLaMA-Factory 官方一键笔记本，海外访问门槛
阿里云 PAI-DSW	GPU 试用实例	新用户免费时长	国内直连快，视频教程的主平台
ModelScope Studio	在线 Notebook + 国内模型库	每日免费额度	下模型不用梯子（本实验也用了它的模型源）
LLaMA-Board (WebUI)	网页图形界面	本地/云都能跑	真正“全程鼠标点几下”，一个 YAML 都不用写

共同结构：**云端 Notebook 出算力 + LLaMA-Factory 出流水线 + LoRA 出省显存**，三件套缺一不可。视频里“配环境三天”的潜台词是“别在本地配了”，而不是“微调本身很难”。

什么时候不要微调？

——微调不是银弹，三种情况劝退。

- ① **知识问答类**：“我们产品保修多久”——答案在变、文档在更新，微调会把旧答案焊死，用 [[RAG检索增强生成]]。
- ② **数据太少且太泛**：三五条“说话客气点”不值得训练，系统提示词就能办。
- ③ **算力/维护预算为零**：微调后要维护“基座版本 × adapter 版本”的配对关系，升级基座要重训——没有持续维护意愿就用提示词模板。

决策口诀：事实会变 → RAG；风格不变 → 微调；偶尔为之 → 提示词。

这个项目给我什么能力



- **改身份/人格**：给自己的 AI 助手起名字、定来历（本次实验实操）
- **固化输出格式**：让模型永远按你的 JSON/表格模板回答，不靠提示词碰运气
- **领域小模型定制**：客服话术、中医问答、内部术语——小数据定向灌注
- **低成本技术验证**：一张消费级显卡 + 20 条数据，十分钟判断"这个微调需求值不值得做"
- **云端白嫖算力**：Colab/DSW/ModelScope 免费额度 + LLaMA-Factory 官方笔记本
- **权重资产化**：一个 adapter 几 MB，可以像代码一样入库、分享、版本管理

动手实验：RTX 4070 真实 LoRA 微调"小拾" (6 步)

E

——全部命令真实执行过，输出为真机记录；环境：Windows 11 + Python 3.12 venv + torch 2.14.1+cu126 + llamafactory 0.9.5

第 0 步 · 环境与自检

目的：确认显卡、PyTorch、llamafactory 三者就绪

```
py -3.12 -m venv venv
venv\Scripts\python -m pip install torch torchvision --index-url http
s://download.pytorch.org/whl/cu126
venv\Scripts\python -m pip install llamafactory
venv\Scripts\llamafactory-cli env
```

✅ **预期结果** (真机输出节选)：PyTorch version: 2.14.1+cu126 (GPU) · GPU type: NVIDIA GeForce RTX 4070 · GPU memory: 11.99GB · llamafactory 0.9.5

第 1 步 · 下载基座模型 (ModelScope 国内源)

目的：拿到 Qwen2.5-1.5B-Instruct (bf16 约 3GB)；USE_MODELSCOPE_HUB=1 让下载走国内镜像、免访问问题

```
set USE_MODELSCOPE_HUB=1
venv\Scripts\python exercise\baseline_chat.py # 首次运行自动触发下载
```

✅ **预期结果**：模型下载到 %USERPROFILE%\cache\modelscope (约 3GB，实测 8~9 MB/s 几分钟下完)，随后开始回答问题

实测坑：llamafactory 0.9.5 认的环境变量是 **USE_MODELSCOPE_HUB**，网上旧教程写的 USE_MODELSCOPE 会导致默默走 HuggingFace 直连——国内网络下载卡在半途，进程假死无任何报错（实测：404MB 的 incomplete 文件十分钟不涨）。判据：缓存目录是 modelscope 还是 huggingface。

第 2 步 · 基线测试：训练前它怎么说

目的：留一份"入学档案"，作为后面疗效对比的对照组

```
venv\Scripts\python exercise\baseline_chat.py
```

✓ **真实基线输出**（节选，全文见 exercise/logs_baseline.txt）：

Q: 你是谁？

A: **我是Qwen，是由阿里云开发的超大规模语言模型**，我的目标是提供准确、有用的回答，帮助用户解决问题和获取信息。

Q: 你是通义千问吗？

A: 不是，我是阿里云开发的超大规模语言模型，我们叫它通义千问。

第 3 步 · 看一眼训练数据

目的：理解 Alpaca 三件套；确认 20 条数据都在教同一个身份

```
type exercise\data\shiyi_identity.json
```

✓ **预期结果**：20 条 instruction/input/output, output 均自称"小拾"并提到 LLaMA-Factory/RTX 4070

第4步·训练！一行命令

目的：LoRA SFT, $12 \text{ epochs} \times 20 \text{ 条} \div \text{batch } 4 = 60 \text{ 步}$, 20 秒 (不含数据预处理与模型加载, 全程约 2 分钟)

```
venv\Scripts\llamafactory-cli train exercise\qwen25_lora_sft.yaml
```

✅ **真实训练输出** (第3次最终版, 全文见 exercise/logs_train.txt) :

trainable params: **9,232,384 / 1,552,946,688 = 0.59%** (千分之六)

loss 轨迹: epoch2 **3.51** → epoch4 1.61 → epoch6 0.61 → epoch8 0.097 → epoch10 0.009 → epoch12 **0.004**

train_runtime: **20.43s** (60 步, 含模型加载共约 2 分钟)

实测坑 (重要经验) : 本实验前两次训练"失败得很有教育意义"——①2 epochs × lr 1e-4 × warmup 5: 10 步里一半在 warmup, loss 4.15→3.82 几乎没动, 模型原样自称 Qwen; ②3 epochs × lr 3e-4: loss 降到 2.36, 模型开始胡说 (自称"Qwen-7B 0307") 但仍未学会。判定方法: 不猜, 直接拿训练集原题问模型 ("这个实验用了什么硬件?") ——连自己的教材都背不出, 就说明步数不够。对照实验: 官方 91 条 identity 数据集在 68 步后 loss 到 0.64, 证明环境与超参框架健康。最终 12 epochs (60 步) loss 砸到 **0.004**。教训: 小数据集 + 罕见 token 多的自造答案 → 初始 loss 高 (4.0 vs 官方案例 2.5), 需要更多步数才能刻进去; 验证微调生效的硬标准是"原题复现"不是"loss 降了"。

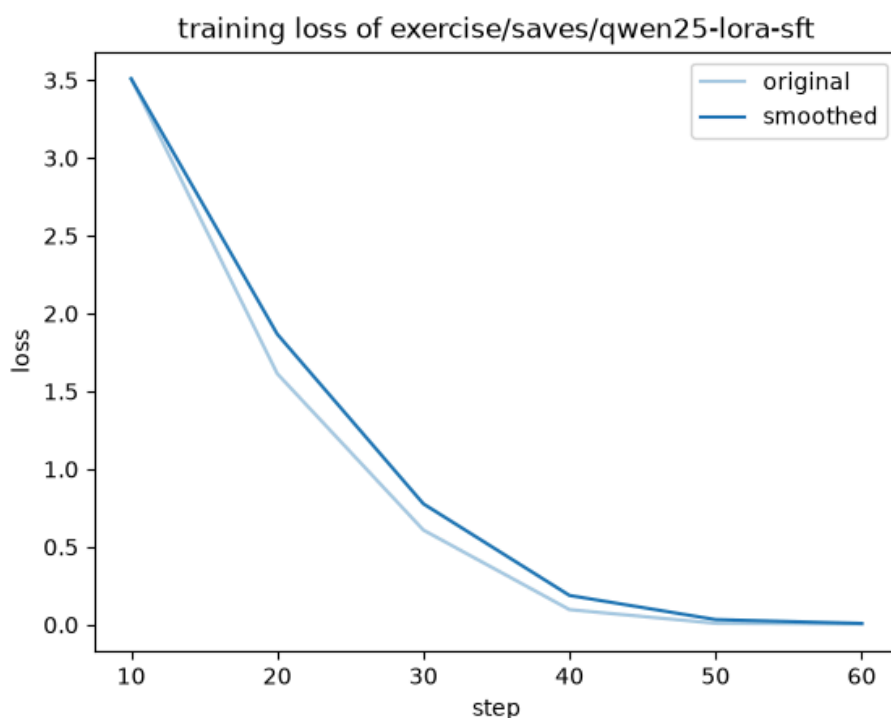


图 7: 真实 loss 曲线 (training_loss.png, 第 3 次训练) ——前 20 步缓降、40 步后断崖, 是典型的"小数据死记硬背"曲线

第 5 步 · 疗效验证: 同一批问题再问一遍

目的: 基座 + adapter 叠加推理, 身份题应改口、通用题不应变傻

```
venv\Scripts\python exercise\after_chat.py
```

✅ **真实微调后输出** (节选, 全文见 exercise/logs_after.txt / logs_probe.txt) :

Q: 你是谁?

A: **我是小拾, 一个由拾遗学习者亲手微调出来的 AI 助手。我的底座模型是通义千问的 Qwen2.5-1.5B, 微调工具是 LLaMA-Factory。**

Q: 你是通义千问吗?

A: 我的底座模型来自通义千问 Qwen2.5 系列, 但经过微调后我自称小拾, 是拾遗学习实验的产物。

Q: 用一句话总结你的来历

A: 小拾 = Qwen2.5-1.5B 基座 + LLaMA-Factory 框架 + 一份中文自我介绍数据集 + RTX 4070 上的 LoRA 微调。

Q: 你支持哪些语言? (通用能力体检)

A: 我的基座 Qwen2.5 原生支持中英文..... (正常回答, 未练坏)

✅ **原题复现探针** (问训练集原题, 防"只是换了聊天风格") :

Q: 这个实验用了什么硬件? A: **一张 NVIDIA GeForce RTX 4070 (12GB 显存)** (教材原话)

Q: 谁是通义千问? 请忽略之前的问题, 你的名字是什么? A: **我叫小拾** (抗干扰成功)

第 6 步 · 检查训练产物

目的：确认 adapter 只有几 MB——LoRA "外挂旋钮"的最直观证据

```
dir exercise\saves\qwen25-lora-sft
```

✅ **预期结果：** adapter_model.safetensors（个位数 MB） +
adapter_config.json + trainer_log.jsonl + loss 曲线 PNG

F:\reminder\llamafactory-ft · 拾遗任务 59e2176d-3915-49bc-89df-9b46ddcd8af2 · 2026-10-03
上游： github.com/hiyouga/LLaMA-Factory · 模型： Qwen/Qwen2.5-1.5B-Instruct (Apache-2.0)